Question 5: Deploy an application to a Kubernetes (Minikube) cluster using Deployment and Service manifests, and verify the running pods.

Course outcome: CO1, CO2

The containerized app was deployed to a Minikube cluster using a ConfigMap, a Deployment (2 replicas with health probes) and a NodePort Service. Pods were verified and the app was reachable through the service.

Manifests

k8s/deployment.yaml

```
# Lab 2 - Deployment for the flask app
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab2-app
  labels:
    app: lab2-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: lab2-app
  template:
    metadata:
      labels:
        app: lab2-app
    spec:
      containers:
        - name: lab2-app
          image: devops-lab2-app:2.0.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
          envFrom:
            - configMapRef:
                name: lab2-app-config
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 15
            periodSeconds: 20
```

k8s/service.yaml

```
# Lab 2 - NodePort service exposing the flask app
apiVersion: v1
kind: Service
metadata:
  name: lab2-app-service
spec:
  type: NodePort
  selector:
    app: lab2-app
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30080
```

k8s/configmap.yaml

```
# Lab 2 - ConfigMap for the flask app
apiVersion: v1
kind: ConfigMap
metadata:
  name: lab2-app-config
data:
  APP_MESSAGE: "DevOps Lab 2 app running on Kubernetes (Minikube)"
```

Getting the image into the cluster

```
$ eval $(minikube docker-env)   # experimental with containerd - FAILED (buildkit 404)
$ minikube image load devops-lab2-app:2.0.0   # supported approach
$ minikube image ls | grep devops-lab2
```

Deploy and verify

```
$ kubectl apply -f k8s/
configmap/lab2-app-config created
deployment.apps/lab2-app created
service/lab2-app-service created

$ kubectl rollout status deployment/lab2-app
deployment "lab2-app" successfully rolled out

$ kubectl get pods -l app=lab2-app -o wide
lab2-app-6b76d4c76f-bdqmj  1/1  Running  10.244.0.4  minikube
lab2-app-6b76d4c76f-spsnb  1/1  Running  10.244.0.3  minikube

$ kubectl get service lab2-app-service
lab2-app-service  NodePort  10.100.128.147  5000:30080/TCP
```

Accessing the app

```
$ curl http://192.168.49.2:30080/health
{"status":"UP","version":"2.0.0"}

$ curl http://192.168.49.2:30080/
{"message":"DevOps Lab 2 app running on Kubernetes (Minikube)","version":"2.0.0"}
```

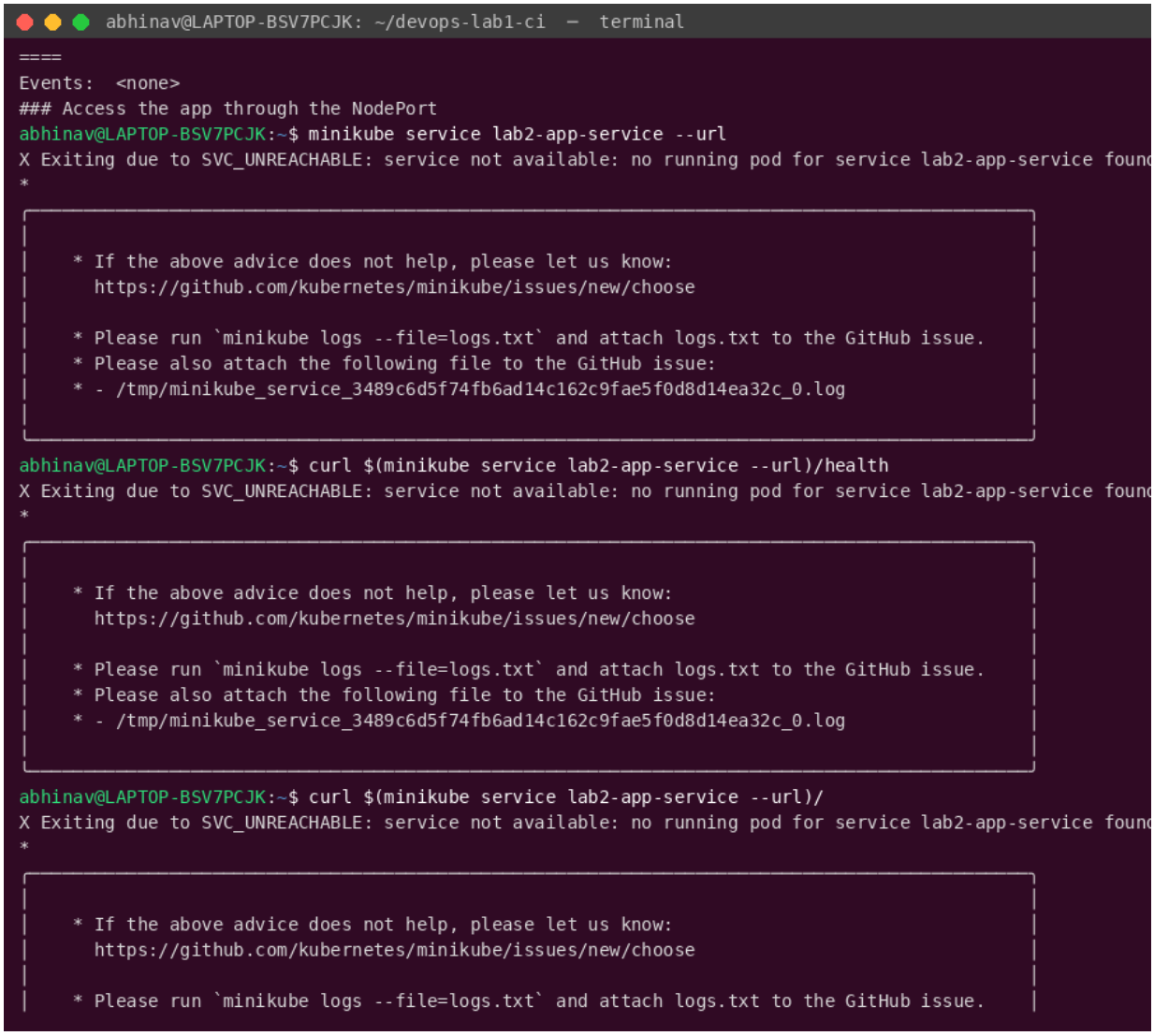


Fig: kubectl apply, rollout status and pod verification



Fig: Application served by the cluster (via kubectl port-forward)

Observations

- Minikube v1.39 on WSL2 required --force-systemd=true (upstream issue #20469); without it the node container exits with code 255 shortly after boot.
- With the containerd runtime, `eval $(minikube docker-env)` builds fail (buildkit 404) - `minikube image load` is the supported way to move images into the cluster.
- The readiness/liveness probes target /health, so a broken deploy never receives traffic.
- NodePort access works from WSL via 192.168.49.2:30080; from Windows, kubectl port-forward to localhost is the convenient path.

Conclusion

All experiments were performed on Ubuntu 24.04 LTS with Docker, Docker Compose, Minikube and kubectl. The evidence (command logs, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-2 under the docs/ directory.